Discovering Daily Electricity Use Patterns with Apache Spark

1. Introduction

Daily electricity totals show how much a user consumes but not when the demand occurs. This project investigates whether historical meter records contain stable and interpretable daily load patterns, and how often each user follows them on weekdays and at weekends. One observation is one user's day, represented by its 24-hour load curve. As planned in the proposal, the data are stored in HDFS and Delta Lake, prepared with PySpark and Spark SQL and clustered with K-means in Spark ML [1].

2. Data

The ElectricityLoadDiagrams20112014 dataset [2] contains 15-minute average power readings (kW) from 370 Portuguese meters between 2011 and 2014: 140,256 timestamps and 51.9 million readings. Each timestamp marks the end of its interval, so every reading was moved back by 15 minutes before it was assigned to a day and an hour; the energy of an hour is the sum of its four readings divided by four. Validation found no duplicate, missing or invalid timestamps and no unparseable or negative readings. Of the 540,570 user-days, 428,691 were valid. The others were excluded because consumption was zero (105,096 days, 98% of them before the meter's first recorded consumption), because at least half of the readings were zero (3,823) or because the day was a daylight-saving transition (2,960), on which the source zero-fills or double-counts one hour. The study uses 2013, when 330 meters have at least 90% valid days, and keeps 2014 as a holdout year.

3. Method

Two Spark stages turn the raw meter file into labelled daily curves. The 711 MB file is stored in HDFS as six blocks. An ingestion stage parses it, unpivots it from one column per meter to one row per reading and writes the validated readings to a Delta table partitioned by year. A feature stage then builds all daily curves in a single aggregation: 24 conditional sums per user and day produce the hourly energy with one shuffle. The same stage labels each day as a weekday, Saturday, Sunday or weekday public holiday (Portuguese national calendar) and flags low days, whose consumption is below half of the user's median valid day. Delta tables, the fitted model and the raw file reside in a single-node HDFS (Hadoop 3.3.6 in Docker); the same code also runs on the local file system. A Plotly report shows the pattern curves, the mix by day of week and per-user calendars.

The clustering compares the shape of each daily curve rather than its size. A Spark ML pipeline assembles each curve into a vector and divides it by the daily total, so that a feature vector describes the share of the day's energy used in each hour. Daily energy is kept for interpretation, and division by the daily peak was evaluated as an alternative. K-means was fitted to the 2013 days of 330 meters with at least 300 valid days, for k = 3 to 8 with five seeds each. The final model is the restart with the lowest within-cluster cost at the chosen k, and it is persisted as a pipeline model. Because Spark's k-means initialisation also depends on row order, every fit reads the same cached feature table and the final model is the sweep's own run.

The evaluation looks beyond silhouette, which cannot show whether patterns distinguish weekdays from weekends. Each configuration was therefore also scored on restart reproducibility (the number of seeds that reach the lowest cost within 0.1%, and the pairwise adjusted Rand index), on the lift of each day type within each cluster and on the conditional mutual information I(cluster; weekend | user), which is zero for a clustering that only separates users. A holiday check considered users whose most frequent Sunday pattern differs from their weekday one and compared how often their weekday holidays and their ordinary weekdays take the Sunday pattern. The 2014 days were assigned with the 2013 model. A separate experiment compared both normalisations for k = 3 to 6 with ten seeds each.

4. Results

Table 1 and Figure 1 summarise the final model, which uses share normalisation and k = 5.

Table 1. Daily patterns in 2013 (119,677 days, 330 meters). Lift is P(day type | pattern) / P(day type).

Pattern	Days	Sunday lift	Holiday lift	Low-day lift
Broad daytime plateau (09–22 h)	50.3%	0.91	0.77	0.11
Evening peak (19–23 h)	18.4%	0.90	0.96	0.56
Flat all day	13.8%	1.56	2.24	5.88
Low-base plateau (11–22 h)	10.2%	1.00	0.86	0.18
Office hours (09–19 h)	7.3%	0.80	0.57	0.21

The flat pattern contains base load only and behaves as a closed-day pattern: low days are 5.9 times and weekday holidays 2.2 times more frequent in it than overall. The office-hours plateau shows the opposite tendency on Sundays (0.80) and on holidays (0.57). No pattern is specific to weekends, since the largest Sunday lift is 1.56.

Reproducibility rather than silhouette decided k. Mean silhouette was highest at k = 4 (0.486) and fell to 0.437 at k = 5 and 0.239 at k = 8. It favours coarse partitions: k = 3 scores 0.461 although it has no closed-day pattern and places low days in the evening cluster. With share normalisation, the flat pattern already appears at k = 4, but in the ten-seed

experiment only 6 of 10 seeds reached the lowest cost and the flat cluster matched across seeds with a mean Jaccard index of 0.73. At k = 5 these values rose to 8 of 10 seeds and 0.91. Peak normalisation reached the lowest cost with all ten seeds at k = 3 and k = 4 and also produced a flat cluster from k = 4 (10.3% of days at k = 5); at k = 5, however, it formed a wider 09–20 h plateau (15.4% of days) instead of the office-hours pattern. A refit with peak normalisation at k = 5 agreed with the final model at an adjusted Rand index of 0.67; the best-match Jaccard index was 0.94 for the low-base plateau and 0.69 for the flat pattern.

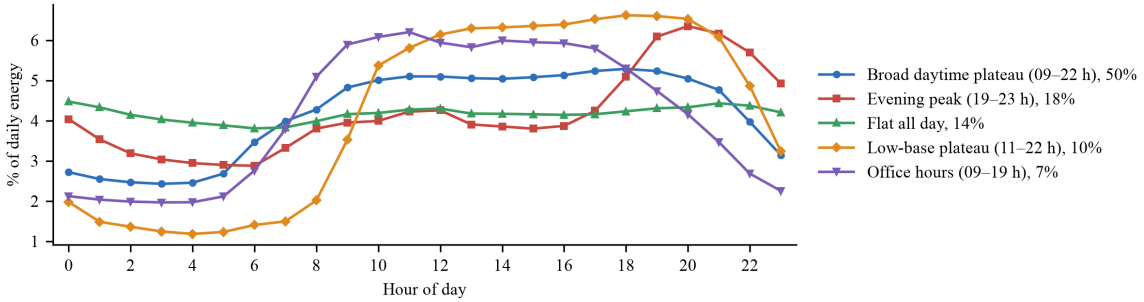


Figure 1. Cluster centres of the final model: share of the daily energy used in each hour.

Weekly structure explains a small part of the variation. The conditional mutual information was 0.071 bits, whereas the weekend indicator carries 0.864 bits, and knowing the user removes 73% of the cluster entropy. Most users keep the same mix of patterns all week (median total-variation distance 0.04 between weekday and weekend mixes), but 79 of 330 users differ by more than 0.2. For the 55 users whose Sunday pattern differs from their weekday pattern, 50% of weekday holidays took the Sunday pattern, compared with 15% of ordinary weekdays. In 2014 the silhouette was 0.444 (0.439 in 2013), every pattern share changed by less than one percentage point, and 95.5% of the user and day-type pairs kept their dominant pattern.

The full pipeline ran in about 80 s on a 14-core laptop with Spark 3.5.6 in local mode: ingestion took 25.5 s, the daily curves 7.2 s, the k sweep 32.8 s and the final model with its checks 8.9 s (one run each). On HDFS the three stages that read or write tables took about 4 s longer (29.8, 11.1 and 12.8 s), the k sweep on cached features took 33.1 s, and the cluster centres and silhouettes were identical.

5. Discussion and Limitations

The 2013 data contain five stable daily patterns, but they mostly characterise users rather than days. Weekly differences arise mainly when a minority of users change pattern on Sundays, most visibly towards the flat pattern. Weekday holidays take these users' Sunday pattern far more often than ordinary weekdays do, which suggests that the patterns capture whether a site operates rather than the calendar alone. Customer types are unknown, however, so names such as office hours only interpret curve shapes.

The analysis covers one study year and one holdout year. The 50% low-day threshold is a modelling choice, municipal holidays are not included, and 50 of 330 users have a different dominant pattern in winter (December to February) than in summer (June to August), a seasonal effect that the profiles do not model. At k = 5, 2 of 10 restarts still end in other local optima, so the result depends on keeping the lowest-cost restart. HDFS ran as a single node, and the times come from one run on one machine; they show the relative cost of the stages, not how the pipeline scales. Finally, one meter with an unusual 13:00–17:00 load block (MT_093) was left out of fitting, because it otherwise occupied a cluster of its own and made the K-means restarts disagree.

6. Running the Code

The archive contains the Python modules, a README that lists all options, and the HDFS configuration. The dataset (711 MB) is not included: it can be downloaded from the UCI repository [2] and unzipped into `data/raw`. With Python 3.11 or later and Java 17, these commands install the dependencies and run ingestion, feature construction, clustering and the profiles:

```
pip install -r requirements.txt
python pipeline.py
```

Delta tables are written to `data/delta`, the model to `data/models`, and the HTML report and run log to `output`. `python pipeline.py explore` repeats the comparison of normalisations and k. To run on HDFS, `docker compose -f hdfs/docker-compose.yml up -d` starts a single-node instance; the README gives the commands that upload the file and point the pipeline at `hdfs://` paths.

References

[1] Apache Software Foundation. Spark 3.5.6: Clustering. Spark documentation.

[2] A. Trindade. ElectricityLoadDiagrams20112014. UCI Machine Learning Repository, 2015. doi:10.24432/C58C86. https://archive.ics.uci.edu/dataset/321/electricityloaddiagrams20112014